

PROJECT REPORT

Personal Expense Tracker

A CLI-Based Expense Management System using Python, SQLite & Matplotlib

Submitted by

**Kathyaini Thota Registration
Number: 26BAC10019 Program:
B.Tech ECE (AI & Cybernetics)**

Course Category: Python Programming Academic Year: 2025 –
2026

Table of Contents

- 1. Introduction
- 2. Project Overview & Objectives
- 3. System Requirements & Technology Stack
- 4. System Architecture & Program Flow
- 5. Database Design
- 6. Module-wise Code Description
- 7. Key Programming & SQL Concepts Demonstrated
- 8. Sample Execution / Example Workflow
- 9. Input Validation & Error Handling
- 10. Testing Observations
- 11. Limitations
- 12. Future Enhancements
- 13. Learning Outcomes
- 14. Conclusion
- 15. Appendix – Full Source Code (Personal Expense Tracker)

1. Introduction

Managing day-to-day personal finances is a challenge many people face because expenses are scattered across cash payments, notes, and memory rather than a single organized record. The **Personal Expense Tracker** is a command-line Python application built to solve this problem in a lightweight, beginner-friendly way. It allows a user to record expenses, persist them in a structured SQLite database, search and review past records, and generate both a numerical and a visual (pie-chart) summary of spending by category.

The project is implemented entirely in Python and demonstrates the integration of several core computer-science concepts – procedural program design, relational data storage using SQL, exception handling, and data visualization – inside a single, coherent application. It is designed as an Python project (Personal Expense Tracker) that is easy to run, easy to extend, and easy to explain in a viva or project demonstration.

2. Project Overview & Objectives

The application provides a simple digital alternative to manual expense tracking. Through a numbered CLI menu, the user can add new expenses, view all stored records, generate a category-wise report with percentage contribution and an accompanying pie chart, search records by date or category, reset all stored data, or exit the program (optionally clearing all data before closing).

2.1 Objectives

- To develop a simple, functional application for managing personal expenses.
- To store expense information persistently using a structured SQLite database.
- To provide an easy-to-use, menu-driven command-line interface.
- To allow users to search and filter expense records by date or category.
- To calculate category-wise total spending and its percentage of overall expenditure.
- To visualize the expense distribution using a Matplotlib pie chart.
- To demonstrate practical, combined use of Python, SQL and data visualization.
- To build a project structure that can be maintained and extended with future features.

3. System Requirements & Technology Stack

The project relies on a small, standard set of tools, all freely available with Python 3.8+.

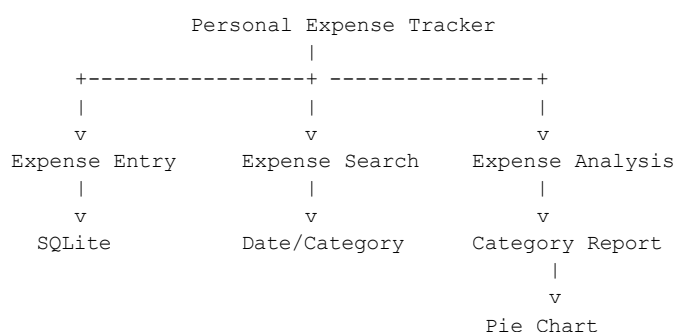
Technology / Module	Role in the Project
Python 3.8+	Core programming language for the entire application
sqlite3 (built-in)	Serverless relational database used to persist expense records
Matplotlib	Generates the category-wise pie chart (chart.png)
SQL (DDL/DML)	CREATE TABLE, INSERT, SELECT, WHERE, LIKE and DELETE operations
Technology / Module	Role in the Project

Command-Line Interface	Menu-driven text interface for all user interaction
Git & GitHub	Recommended for version control and project hosting

4. System Architecture & Program Flow

The application follows a simple, modular, function-based architecture. Each menu option maps to one independent Python function, and all functions share a single SQLite connection/cursor created at program start. This keeps the design easy to follow while still separating concerns cleanly.

4.1 Functional Modules



4.2 Program Flow

```

START
|
v
Connect to expenses.db -> CREATE TABLE IF NOT EXISTS expenses
|
v
Display Main Menu (loop)
|
v
Get User Choice
|
+-- 1: Add Expense          -> INSERT INTO expenses
+-- 2: View Expenses        -> SELECT * FROM expenses
+-- 3: Report + Pie Chart    -> aggregate totals -> plt.savefig('chart.png')
+-- 4: Search Expenses      -> SELECT ... WHERE date/category LIKE ?
+-- 5: Exit                 -> optional DELETE ALL -> conn.close() -> END
+-- 6: Reset All Data       -> DELETE FROM expenses (with confirmation)
|
v
(loop back to menu unless Exit was chosen)
  
```

5. Database Design

The application uses a single SQLite database file, **expenses.db**, containing one table, **expenses**. SQLite is used because it is serverless, file-based, and ships with Python's standard library (`sqlite3`), which removes the need for any external database server or installation step.

5.1 Table Schema – expenses

Column	Type	Description
date	TEXT	Date the expense occurred, entered as DD-MM-YYYY
category	TEXT	Expense category, e.g. Food / Travel / Study / Other
amount	REAL	Monetary amount spent, stored as a floating-point number

Table creation statement used at program start (executed only if the table does not already exist):

```
CREATE TABLE IF NOT EXISTS expenses (
    date TEXT,
    category TEXT,
    amount REAL
)
```

5.2 Example Record

Date: 26-09-2026
Category: Food
Amount: 150

6. Module-wise Code Description

The Personal Expense Tracker source code is organized into seven functions. Each function handles a specific part of the application, making the program easier to understand and maintain. The following sections explain what each function does and how it works.

Function	Purpose
add_expense()	Prompts for date, category and amount, validates the amount, and inserts the record into the database
view_expenses()	Retrieves and prints every stored expense record
generate_report()	Computes the overall total and per-category totals/percentages, prints a text report, and saves a pie chart as chart.png
search_expenses()	Searches records where the date or category matches a user-supplied keyword using SQL LIKE
reset_data()	Deletes all expense records after an explicit y/n confirmation
main_menu()	Displays the menu in a loop, reads the user's choice, and dispatches to the corresponding function

6.1 Database Initialization

At import time the script opens a connection to `expenses.db` and ensures the `expenses` table exists, using a single shared `cur` (cursor) and `conn` (connection) object that every function below reuses.

```
conn = sqlite3.connect("expenses.db")
cur = conn.cursor()

cur.execute("""
Create table if not exists expenses (
    date TEXT,
    category TEXT,
    amount REAL

```

```
)
"""
conn.commit()
```

6.2 add_expense()

Collects three inputs from the user: date, category and amount. The amount is converted to a float inside a `try/except` block; if conversion fails, an error message is printed and the function returns early without touching the database. On success, the new row is inserted using a parameterised query (protecting against SQL injection) and the transaction is committed.

```
def add_expense():
    date = input("Enter date (DD-MM-YYYY): ")
    category = input("Enter the category of expenditure (Food/Travel/Study/Other): ")
    try:
        amount = float(input("Enter amount: "))
    except ValueError:
        print("Invalid amount. Please enter a number.")
        return

    cur.execute("Insert into expenses values (?, ?, ?)", (date, category, amount))
    conn.commit()
    print("Expense added!")
```

6.3 view_expenses()

Runs an unfiltered `SELECT * FROM expenses` query and prints every row returned, formatted with the rupee symbol for readability.

```
def view_expenses():
    cur.execute("SELECT * FROM expenses")
    rows = cur.fetchall()
    print("\n--- All Expenses ---")
    for row in rows:
        print(f>Date: {row[0]}, Category: {row[1]}, Amount: Rs.{row[2]}")
```

6.4 generate_report()

This is the analytical core of the project. It fetches every (amount, category) pair, computes the grand total, and builds a dictionary (`category_totals`) mapping each category to its cumulative amount. It then prints the total and, for each category, the amount and its percentage share of the total (guarding against division by zero when there are no records). Finally, if there is at least one category, it draws a Matplotlib pie chart of the category totals and saves it as `chart.png`.

```
def generate_report():
    cur.execute("SELECT amount, category FROM expenses")
    rows = cur.fetchall()

    total = sum(r[0] for r in rows)
    category_totals = {}

    for amount, category in rows:
        category_totals[category] = category_totals.get(category, 0) + amount

    print("\n--- Expense Report ---")
    print(f>Your Total Expenses: Rs.{total}")
    for c, amt in category_totals.items():
        percent = (amt / total) * 100 if total > 0 else 0
        print(f">{c}: Rs.{amt} ({percent:.2f}%)")

    if category_totals:
        plt.figure(figsize=(6,6))
        plt.pie(category_totals.values(), labels=category_totals.keys(), autopct='%1.1f%%')
        plt.title("Expense Distribution by Category")
        plt.savefig("chart.png")
```

```
plt.close()
print("Pie chart saved as 'chart.png'. Open it to view.")
```

6.5 search_expenses()

Accepts a single keyword and searches both the `date` and `category` columns using SQL `LIKE` with wildcard padding (`%keyword%`), so a partial date or partial category name is enough to find a match. If no rows are returned, the user is informed that nothing matched.

```
def search_expenses():
    key = input("Search by date or category: ")
    cur.execute("SELECT * FROM expenses WHERE date LIKE ? OR category LIKE ?", (f"%{key}%", f"%{key}%"))
    rows = cur.fetchall()

    if rows:
        print(f"\n--- Search Results for '{key}' ---")
        for r in rows:
            print(f>Date: {r[0]}, Category: {r[1]}, Amount: Rs.{r[2]}")
    else:
        print("Result not found.")
```

6.6 reset_data() and Exit-time Cleanup

`reset_data()` asks for an explicit `y/n` confirmation before executing `DELETE FROM expenses`, preventing accidental data loss. A similar confirmation is repeated inline in the exit branch of the menu (choice 5), letting the user optionally wipe all records before the program closes the database connection.

```
def reset_data():
    confirm = input("Are you sure you want to delete ALL expenses? (y/n): ")
    if confirm.lower() == "y":
        cur.execute("DELETE FROM expenses")
        conn.commit()
        print("All expenses cleared!")
    else:
        print("Reset cancelled.")
```

6.7 main_menu()

Implements an infinite loop that prints the menu, reads the user's numeric choice as a string, and dispatches to the matching function via a chain of `if/elif` statements. Choice 5 (Exit) additionally offers to clear all data, closes the database connection, and breaks out of the loop; any unrecognised choice triggers a re-prompt rather than crashing the program.

```
def main_menu():
    while True:
        print("\nExpense Tracker Menu")
        print("1. Add Expense")
        print("2. View Expenses")
        print("3. Report (with Pie Chart)")
        print("4. Search Expenses")
        print("5. Exit")
        print("6. Reset all Data")

        choice = input("Enter choice: ")
        if choice == "1": add_expense()
        elif choice == "2": view_expenses()
        elif choice == "3": generate_report()
        elif choice == "4": search_expenses()
        elif choice == "5":
            confirm = input("Do you want to delete all expenses before exiting? (y/n): ")
            if confirm.lower() == "y":
                cur.execute("DELETE FROM expenses")
                conn.commit()
                print("All expenses deleted.")
```

```

conn.close()
print("Good Work. Goodbye!")
break
elif choice == "6": reset_data()
else:
    print("Invalid choice. Try again.")

```

7. Key Programming & SQL Concepts Demonstrated

7.1 Python Concepts

- **Variables** – storing dates, categories, amounts and totals.
- **Functions** – each menu feature is isolated in its own function (add_expense, view_expenses, search_expenses, generate_report, reset_data).
- **Conditional statements** – the if/elif chain used for menu dispatch and input checks.
- **Loops** – the while True menu loop and for loops used while printing rows and building category totals.
- **Exception handling** – a try/except ValueError block guards against non-numeric expense amounts.
- **Dictionaries** – category_totals accumulates per-category sums using dict.get() with a default value.
- **Modules** – sqlite3 for persistence and matplotlib.pyplot for visualization.

7.2 SQL Concepts

Clause / Statement	Where it is used
CREATE TABLE IF NOT EXISTS	Initializing the expenses table on first run
INSERT INTO ... VALUES (?, ?, ?)	add_expense() – parameterised insert of a new record
SELECT * / SELECT amount, category	view_expenses() and generate_report()
WHERE ... LIKE ? OR ... LIKE ?	search_expenses() – flexible substring search
DELETE FROM expenses	reset_data() and the exit-time cleanup option

8. Sample Execution / Example Workflow

A typical user session proceeds as follows:

Step 1 – Add an expense

```

Enter date (DD-MM-YYYY): 26-09-2026
Enter the category of expenditure (Food/Travel/Study/Other): Food
Enter amount: 150
Expense added!

```

Step 2 – Add another expense

```

Enter date (DD-MM-YYYY): 26-09-2026
Enter the category of expenditure (Food/Travel/Study/Other): Travel
Enter amount: 80
Expense added!

```

Step 3 – View all expenses

```
--- All Expenses ---
Date: 26-09-2026, Category: Food, Amount: Rs.150.0
Date: 26-09-2026, Category: Travel, Amount: Rs.80.0
```

Step 4 – Generate the report

```
--- Expense Report ---
Your Total Expenses: Rs.230.0
Food: Rs.150.0 (65.22%)
Travel: Rs.80.0 (34.78%)
Pie chart saved as 'chart.png'. Open it to view.
```

The generated **chart.png** visually represents the percentage share of each category, making it easy to see at a glance which category (Food, in this case) accounts for the largest portion of total spending.

9. Input Validation & Error Handling

The program validates the expense amount at the point of entry: the raw string typed by the user is converted with `float()` inside a `try/except ValueError` block, so a non-numeric entry results in a friendly error message instead of an unhandled crash. Invalid main-menu choices are also handled gracefully – any option outside 1–6 falls into the `else` branch and re-displays the menu rather than terminating the program.

9.1 Recommended Additional Validation

The README identifies further validation that strengthens the project and can be added as an enhancement:

- Reject amounts that are zero or negative, not just non-numeric input.
- Validate that the date string actually matches the DD-MM-YYYY format (e.g. using the `datetime` module).
- Reject an empty category string instead of storing it as-is.
- Wrap database operations in their own `try/except` blocks to handle SQLite errors safely.

10. Testing Observations

Manual testing of the current Personal Expense Tracker implementation highlighted a few points that are useful to note when using, improving, or evaluating the project:

- Records are stored without a unique identifier (no auto-incrementing id column), so individual rows cannot currently be updated or deleted one at a time – only view/search of all rows and full-table delete are supported.
- `generate_report()` will raise a `ZeroDivisionError`-safe result only because of the 'if total > 0' guard; however, if the expenses table is completely empty, `sum()` over an empty list still returns 0 correctly, and the pie chart step is skipped since `category_totals` is empty.
- The amount is stored and displayed as a raw float, so values may show trailing decimals (e.g. Rs.150.0) rather than being rounded to two decimal places.
- `search_expenses()` matches partial text across both columns simultaneously, so a numeric-looking keyword could coincidentally match part of a date as well as an unrelated category.
- The exit option (5) and the dedicated reset option (6) both offer to delete all data, which is slightly redundant but does give the user two independent confirmation points before permanent data loss.

11. Limitations

As documented in the project README, the current version is intentionally a lightweight CLI application and has the following known limitations:

- No graphical user interface – interaction is entirely text-based.
- No online or cloud synchronization; all data is stored locally in `expenses.db`.
- No user authentication or multi-user support.
- No automatic budget recommendations or spending alerts.
- No update/edit operation for existing records (CRUD's 'Update' is not yet implemented).

12. Future Enhancements

The project can be extended along several directions:

- Monthly and yearly expense tracking with date-based grouping.
- Income tracking alongside expenses for a full personal-finance view.
- Additional chart types – bar charts and line graphs for spending trends over time.
- Budget-limit notifications when a category exceeds a set threshold.
- Support for multiple payment methods (cash, card, UPI, etc.).
- User login and authentication for multi-user use.
- A graphical user interface built with Tkinter.
- A web version built with Flask or Django, and eventually a mobile application.
- Exporting reports to CSV or PDF.
- Cloud database integration and AI-based spending analysis with personalized budgeting suggestions.

13. Learning Outcomes

Completing this project reinforces practical experience with the following areas:

- Core Python programming: functions, conditionals, loops and exception handling.
- File and database management using SQLite.
- Writing and using SQL queries (INSERT, SELECT, WHERE, LIKE, DELETE).
- CRUD-style thinking – Create, Read, Delete are implemented; Update is identified as a future addition.
- Data filtering, aggregation and basic statistical analysis (totals and percentages).
- Data visualization using Matplotlib.
- Command-line application design and menu-driven program flow.
- Use of Git and GitHub for version control and project documentation.

14. Conclusion

The Personal Expense Tracker meets its main objectives by providing a simple command-line application for recording, viewing, searching, and analysing personal expenses. The data is stored in an SQLite database, while Matplotlib is used to create a visual summary of spending. The current version is intentionally simple and does not yet include a graphical interface, multi-user support, or an option to update individual records. Because the program is organized into separate functions, these features can be added later without changing the basic structure of the application. The project can therefore be developed further into a GUI, web application, or more complete personal finance management system.

15. Appendix – Full Source Code (Personal Expense Tracker)

The complete, unmodified source code of the application is reproduced below for reference.

```
import sqlite3
import matplotlib.pyplot as plt

# Connect to SQLite database
conn = sqlite3.connect("expenses.db")
cur = conn.cursor()

# Create table if not exists
cur.execute("""
Create table if not exists expenses (
    date TEXT,
    category TEXT,
    amount REAL
)
""")
conn.commit()

def add_expense():
    date = input("Enter date (DD-MM-YYYY): ")
    category = input("Enter the category of expenditure (Food/Travel/Study/Other): ")
    try:
        amount = float(input("Enter amount: "))
    except ValueError:
        print("Invalid amount. Please enter a number.")
        return

    cur.execute("Insert into expenses values (?, ?, ?)", (date, category, amount))
    conn.commit()
    print("Expense added!")

def view_expenses():
    cur.execute("SELECT * FROM expenses")
    rows = cur.fetchall()
    print("\n--- All Expenses ---")
    for row in rows:
        print(f>Date: {row[0]}, Category: {row[1]}, Amount: {row[2]}")

def generate_report():
    cur.execute("SELECT amount, category FROM expenses")
    rows = cur.fetchall()

    total = sum(r[0] for r in rows)
    category_totals = {}

    for amount, category in rows:
        category_totals[category] = category_totals.get(category, 0) + amount

    print("\n--- Expense Report ---")
    print(f>Your Total Expenses: {total}")
    for c, amt in category_totals.items():
        percent = (amt / total) * 100 if total > 0 else 0
        print(f">{c}: {amt} ({percent:.2f}%)")

    # Pie chart of added expenses
    if category_totals:
        plt.figure(figsize=(6,6))
        plt.pie(category_totals.values(), labels=category_totals.keys(), autopct='%1.1f%%')
        plt.title("Expense Distribution by Category")
        plt.savefig("chart.png")
        plt.close()
        print("Pie chart saved as 'chart.png'. Open it to view.")

def search_expenses():
```

```

key = input("Search by date or category: ")
cur.execute("SELECT * FROM expenses WHERE date LIKE ? OR category LIKE ?", (f"%{key}%", f"%{key}%"))
rows = cur.fetchall()

if rows:
    print(f"\n--- Search Results for '{key}' ---")
    for r in rows:
        print(f>Date: {r[0]}, Category: {r[1]}, Amount:{r[2]}")
else:
    print("Result not found.")

def reset_data():
    confirm = input("Are you sure you want to delete ALL expenses? (y/n): ")
    if confirm.lower() == "y":
        cur.execute("DELETE FROM expenses")
        conn.commit()
        print("All expenses cleared!")
    else:
        print("Reset cancelled.")

def main_menu():
    while True:
        print("\nExpense Tracker Menu")
        print("1. Add Expense")
        print("2. View Expenses")
        print("3. Report (with Pie Chart)")
        print("4. Search Expenses")
        print("5. Exit")
        print("6. Reset all Data")

        choice = input("Enter choice: ")
        if choice == "1": add_expense()
        elif choice == "2": view_expenses()
        elif choice == "3": generate_report()
        elif choice == "4": search_expenses()
        elif choice == "5":
            confirm = input("Do you want to delete all expenses before exiting? (y/n): ")
            if confirm.lower() == "y":
                cur.execute("DELETE FROM expenses")
                conn.commit()
                print("All expenses deleted.")
                conn.close()
                print("Good Work. Goodbye!")
                break
            elif choice == "6": reset_data()
        else:
            print("Invalid choice. Try again.")

if __name__ == "__main__":
    main_menu()

```